AUDIT LOGGING PRIMER · AZURE + DATABRICKS

Audit logging in Azure Databricks: the audit system table versus diagnostic log delivery

Audit logs record who did what: every sign-in, grant, query and secret read. Databricks offers two ways to receive the same events, and most organisations use both.

> LEARNING OBJECTIVE
>
> Describe audit logging in Azure Databricks and the difference between the audit system table and diagnostic log delivery.

01 Why: what goes wrong without audit logs

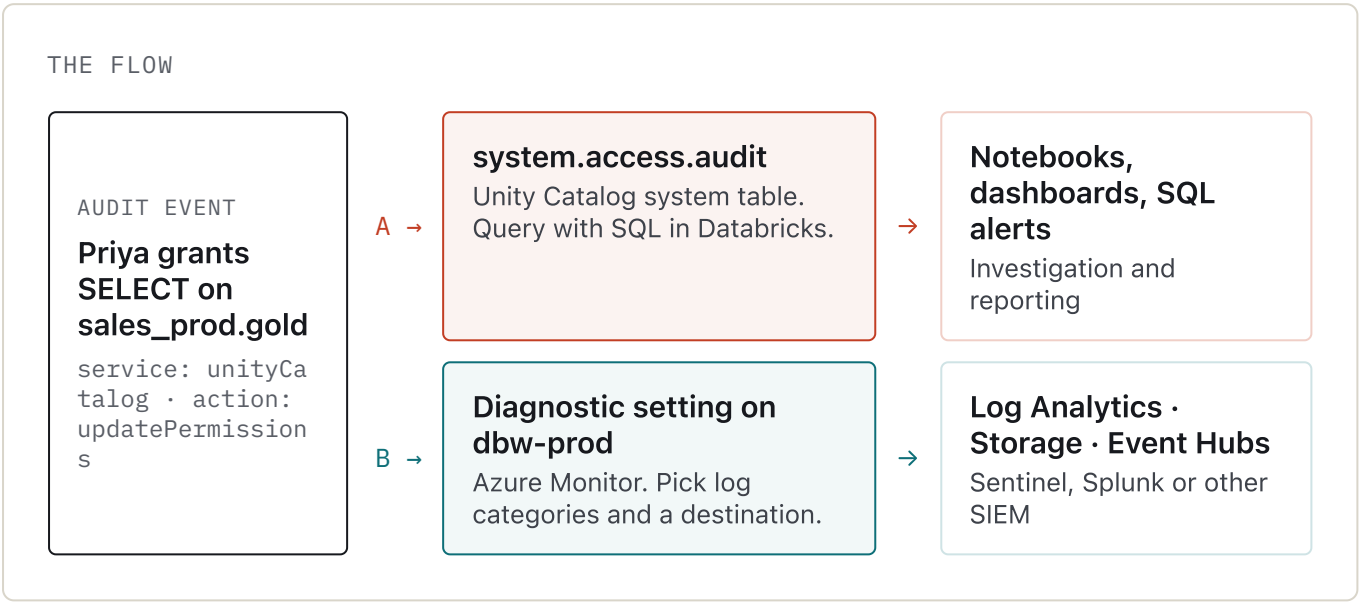
Why audit logs?

Permissions say who *could* act. Audit logs record who *did*: every sign-in, grant, query and secret read. They are the evidence for auditors and the first thing you need in an incident.

RISK	PROTECTION
Data leaks and nobody can say who accessed it	**Access events.** Every table read, grant and download is recorded with identity and IP.
An admin grants themselves access and removes it afterwards	**Permission change events.** The grant and the revoke are both logged permanently.
Compromised credentials are used from an unknown location	**Sign-in events + alerts.** Failed and unusual sign-ins raise alerts in real time.
Regulation requires years of evidence	**Retention you control.** Logs delivered to your own storage or Log Analytics.

02 Audit logging: two ways to get the same events

Databricks records the same audit events either way. You choose how to receive them: query them inside Databricks from a **system table**, or have Azure deliver them to your own tools through **diagnostic settings**. Most organisations use both.



	A · AUDIT SYSTEM TABLE	B · DIAGNOSTIC LOG DELIVERY
Configured in	Databricks. Account admin grants access to the system.access schema.	Azure portal, ARM or Terraform: a diagnostic setting on each workspace resource.
Scope	All workspaces in the account, plus account-level events (e.g. account console changes).	One workspace per setting. Account-level events are not included.
Requires	Unity Catalog-enabled workspace	Premium plan
Latency	Typically within an hour; not for real-time alerting	Minutes; suitable for alerting
Retention	365 days, managed by Databricks	You decide: Log Analytics retention and archive, or storage lifecycle rules
Query with	SQL in notebooks, dashboards and SQL alerts	KQL in Log Analytics; any tool for storage or Event Hubs
Who controls it	Data platform team, inside Databricks	Security team, outside Databricks. Admins in Databricks cannot tamper with it
Cost	No extra charge to store; compute to query	Log Analytics ingestion and retention charges
Best for	Investigation, usage analysis, joining with lineage and billing tables	SIEM integration, real-time alerts, long-term regulatory retention

KEY COLUMNS · SYSTEM.ACCESS.AUDIT	
event_time / event_date	When it happened (UTC)
workspace_id	Which workspace; 0 for account-level events
user_identity.email	Who: user or service principal
service_name · action_name	What: e.g. unityCatalog · updatePermissions
request_params	Details: object names, scope, changes
source_ip_address	From where
response.status_code	Result: 200 success, 403 denied

Turn on verbose audit logs

A workspace setting that adds the text of every notebook command and SQL query to the audit events. Without it you can see that a notebook ran, but not what it did. It applies to both delivery methods.

Recommended: use both

Send diagnostic logs from every workspace to Log Analytics or Sentinel for real-time alerting and long retention the security team controls. Give data platform admins SELECT on `system.access` for investigation, usage reports and account-level events.

03 Which log answers this question?

Five audit questions, each answered with both queries.

Who changed permissions?

Every GRANT and REVOKE in Unity Catalog is logged as updatePermissions, with the object and the changes requested.

A · SQL ON SYSTEM.ACCESS.AUDIT

```
SELECT event_time, user_identity.email,
       request_params.securable_full_name,
       request_params.changes
FROM system.access.audit
WHERE service_name = 'unityCatalog'
  AND action_name = 'updatePermissions'
  AND event_date >= current_date() - 30
ORDER BY event_time DESC;
```

B · KQL IN LOG ANALYTICS

```
DatabricksUnityCatalog
| where TimeGenerated > ago(30d)
| where ActionName == "updatePermissions"
| project TimeGenerated, Identity,
          RequestParams, Response
| order by TimeGenerated desc
```

Best source: Either. Use B to alert the security team in real time; use A to review changes across all workspaces at once.

Who accessed a sensitive table?

Audit events show who requested a table and its credentials. For which queries actually read it, join with the lineage system table.

A · SQL ON SYSTEM.ACCESS.AUDIT

```
SELECT event_time, user_identity.email,
       action_name, source_ip_address
FROM system.access.audit
WHERE service_name = 'unityCatalog'
  AND action_name IN ('getTable',
      'generateTemporaryTableCredential')
  AND request_params.full_name_arg =
      'sales_prod.gold.customers'
  AND event_date >= current_date() - 7;
```

B · KQL IN LOG ANALYTICS

```
DatabricksUnityCatalog
| where TimeGenerated > ago(7d)
| where ActionName in ("getTable",
    "generateTemporaryTableCredential")
| where RequestParams has
    "sales_prod.gold.customers"
| project TimeGenerated, Identity,
          ActionName, SourceIPAddress
```

Best source: A. It sits next to system.access.table_lineage, which records the tables each query read and wrote.

Failed sign-ins

Sign-in attempts to a workspace are logged by the accounts service. A spike of failures from one IP suggests a guessed or stolen credential.

A · SQL ON SYSTEM.ACCESS.AUDIT

```
SELECT event_time, user_identity.email,
       source_ip_address, action_name,
       response.status_code
FROM system.access.audit
WHERE service_name = 'accounts'
  AND action_name LIKE '%Login'
  AND response.status_code <> 200
  AND event_date >= current_date() - 1;
```

B · KQL IN LOG ANALYTICS

```
DatabricksAccounts
| where TimeGenerated > ago(1d)
| where ActionName endswith "Login"
| extend status = toint(
    parse_json(Response).statusCode)
| where status != 200
| summarize failures = count()
    by SourceIPAddress, bin(TimeGenerated, 15m)
```

Best source: B, with an alert rule. Also check Entra ID sign-in logs, where Conditional Access blocks are recorded.

Who read a secret?

Each call to dbutils.secrets.get is logged as getSecret with the scope and key name. The value is never logged.

A · SQL ON SYSTEM.ACCESS.AUDIT

```
SELECT event_time, user_identity.email,
       request_params.scope,
       request_params.key
FROM system.access.audit
WHERE service_name = 'secrets'
  AND action_name = 'getSecret'
  AND request_params.scope = 'kv-erp-prod';
```

B · KQL IN LOG ANALYTICS

```
DatabricksSecrets
| where ActionName == "getSecret"
| where RequestParams has "kv-erp-prod"
| project TimeGenerated, Identity,
          RequestParams
```

Best source: Either. For Key Vault-backed scopes, Key Vault's own diagnostic logs record the read as well.

Compliance profile alerts

On workspaces with the compliance security profile, the monitoring agents on cluster hosts report detections as audit events.

A · SQL ON SYSTEM.ACCESS.AUDIT

```
SELECT event_time, workspace_id,
       service_name, action_name,
       request_params
FROM system.access.audit
WHERE service_name IN (
    'capsule8-alerts-dataplane',
    'clamAVScanService-dataplane')
  AND event_date >= current_date() - 7;
```

B · KQL IN LOG ANALYTICS

```
// Enable the matching log categories
// in the workspace diagnostic setting
DatabricksCapsule8Dataplane
| union DatabricksClamAVScan
| where TimeGenerated > ago(7d)
| project TimeGenerated, ActionName,
          RequestParams
```

Best source: B routed to Sentinel, so a malware detection reaches the security operations team as an incident.

Deliver logs outside Databricks

Diagnostic settings send events to a Log Analytics workspace the security team owns, where workspace admins can't change them.

Query the system table for investigations

system.access.audit covers every workspace and account-level events, and joins with lineage and billing tables.

Logs are only useful if someone looks

Turn on verbose logging, deliver logs to the security team, and set alerts on admin changes, grants and failed sign-ins.